Editorial

Tasks, test data and solutions were prepared by: Vito Anić, Marin Kišić, Dorijan Lendvaj, Martina Licul, Bartol Markovinović, and Patrik Pavić.

Implementation examples are given in attached source code files.

Task Tramvaji

Prepared by: Vito Anić
Necessary skills: arrays, finding minimum

Let us calculate the time it is required to go from station 1 to station x. Let us store those values in the array a. Obviously, from station 1 to station 1 we need 0 minutes. For any other station $x > 1$:

- If the information about the station is `Patrik t`$_i$, then $a_x = t_x$
- If the information about the station is `Josip y t`$_i$, then $a_x = a_y + t_x$ (the time it took from 1 to y plus the time it took from y to x)

This will help us in finding the shortest ride. The shortest ride will always be on adjacent stations, so the indices will be x and $x + 1$. For some station x, the time it is required to go from x to $x + 1$ is $a_{x+1} - a_x$ (if we subtract the time it took from 1 to x, from the time it took from 1 to $x + 1$, we will get the time it took from x to $x + 1$), now we just need to go through the sequence and find the x for which that time is the smallest.

Task Ekspert

Prepared by: Marin Kišić i Martina Licul
Necessary skills: mathematics, bitwise operations

Suppose $y < x$. The product $x \cdot y$ can be represented as the sum of at most $\log_2 y$ numbers. We can do that in the following way:

$x \cdot y = \sum 2^i \cdot x$ for each i where the i-th position in the binary representation of y is 1.

For example: $14 \cdot 13 = 2^0 \cdot 14 + 2^2 \cdot 14 + 2^3 \cdot 14$

Such a solution uses at most $2 \log_2 y$ operations: $\log_2 y$ for storing the result in a register, and $\log_2 y$ for auxiliary operations.

For implementation details, consult the official solution.

Task Lampice

Prepared by: Dorijan Lendvaj i Patrik Pavić
Necessary skills: xor hashing

For the first subtask, let's look at rectangles which contain the point $(0, 0)$ and the ones that don't separately.

If the rectangle contains the point $(0, 0)$, then it must also contain all points (x_2, y_2), and for it to have an area different from 0 it must also contain the point $(1, 1)$. If it has a width of x a height of y, we get that for all points (x_2, y_2) $x \geq x_2$ and $y \geq y_2$. We also know $x \leq n$ and $y \leq m$, so if we add 1 to x_2 and y_2 we get the condition $max(x_2) \leq x \leq n$ and $max(y_2) \leq y \leq m$, so we have $n + 1 - max(x_2)$ possibilities for x and $m + 1 - max(y_2)$ possibilities for y, which is in total $(n + 1 - max(x_2))(m + 1 - max(y_2))$ possible rectangles.

If the rectangle doesn't contain the point $(0, 0)$, then it can't contain any other point (x_2, y_2), so our goal is to find the number of rectangles without any of the chosen points. If we fix the lower row of the rectangle, then for each column we can calculate the maximum height of a rectangle in that column(i.e. how far up can we go without reaching a lamp). If we fix the leftmost column that has the lowest maximum height(let's call it i), we can calculate the number of such rectangles by finding the rightmost column left of i which has a lower or equal maximum height(let's call it l; let it be -1 if there is no such column) and the leftmost column right of i which has a lower maximum height(let's call it r; let it be $m + 1$ if there is no such column), then we know that the height must be between 1 and the maximum height for column i(let's call it h), that the leftmost column of the rectangle is between $l + 1$ and i and that the rightmost column of the rectangle is between i and $r - 1$. So, for the leftmost column we have $i - l$ options, for the rightmost column we have $r - i$ options, but we must subtract the option that the rectangle has a width of 0, for the height we have h options, so the number of rectangles that we're looking for is $h((i - l)(r - i) - 1)$. Quickly calculating l and r for each i is a well known problem that can be solved using a stack, and h can quickly be calculated for each i if we know what its value was for the last row, so the time complexity ends up being $O(nm + k)$.

For the second subtask, we can just check all rectangles and for each color of lamps see if the condition is fulfilled or not, which works in a time complexity of $O(k(nm)^2)$.

For the third subtask, let's uniformly randomly generate an 60-bit number for each color of lamps where all of the generated numbers are independent, and let's give each lamp the value equal to the number generated for its color. If we look at the xor of the values of all the lamps in a rectangle, for each color, if both lamps of that color are in the rectangle, then, since $x \; xor \; x = 0$, that color won't contribute to the xor, and that's also obviously true for colors that aren't included in the rectangle. Therefore, if for each color either both lamps of that color are in the rectangle or both aren't, then the xor of all values in that rectangle will be 0. If it's not, then for each color which has exactly 1 lamp of its color in the rectangle the value is xored with a uniformly randomly distributed number between 0 and $2^{60} - 1$; the result of that is another uniformly randomly distributed number between 0 and $2^{60} - 1$, so the probability that the xor ends up being 0 is $\frac{1}{2^{60}}$. Since there are $\frac{n(n+1)m(m+1)}{4}$ rectangles in total, the probability of this solution saying that a rectangle is good even though it isn't is at most $\frac{n(n+1)m(m+1)}{2^{62}}$, which is small enough. The xor of all values in the rectangle can easily be calculated using prefix sums, which gives us a complexity of $O((nm)^2 + k)$.

For the fourth subtask, if we fix the leftmost and rightmost column of the rectangle(let's call them l and r), our goal is to find the number of rectangles with xor 0 with their leftmost column being l and their rightmost column being r. If we say that a_i is equal to the xor of all values in the i-th row and the l-th to r-th column, our goal is to find the number of subarrays of a_i whose xor is 0 and whose length is at least 2(the second condition can be solved by erasing subarrays of size 1, so we'll ignore them from now on). Let $p_i = xor(a_{0..i-1})$. Then $xor(a_{l..r}) = p_l \; xor \; p_{r+1}$, so we're looking for the number of pairs in the array p whose xor is 0, so we're looking for the number of pairs of equal elements in the array p, which can easily be solved in $O(m \; log \; m)$. The total time complexity is $O(n^2 m \; log \; m)$.

Task Prijateljice

Prepared by: Marin Kišić
Necesssary skills: game theory, dynamic programming

Note that if both Leona and Zoe for each letter of the alphabet have at least one word beginning with that letter, then the winner is the player which has the lexicographically greatest word.

That leads us to the following question: what happens if both have words that begins with the first x letters of the alphabet, but only one of them have a word beginning with the $x + 1$-th letter?

Without loss of generality, suppose Leona has a word beginning with the $x + 1$-th letter. Note that if Leona has the lexicographically greatest word beginning with one of the first x letters, then she is the winner. But if Zoe has such a word, then the game continues.

Now, we can conclude the winner is determined by the first letter which a player doesn't have a word beginning with, and the other player has the lexicographically greatest word among all the words beginning with the letters before that letter.

Alternatively, the task can be solved with dynamic programming. This is left as an exercise for the reader.

Task Kruhologija

Prepared by: Patrik Pavić
Necessary skills: dfs, basic topology

We can use the Euler characteristic of the surface, mainly $V - E + F = 2 - 2g$ where g is the number of holes. Now we have to count V, E and F. To count F we can simply do a dfs on the faces, marking each face we visit. The number of visited nodes is exactly F. Because each face has degree exactly 4, by the handshaking lemma $E = 2F$.

The trickiest part is counting the vertices. For each face you can actually calculate the degrees of the four vertices forming the side. Let's say you are trying to calculate the degree of the left vertex of the edge you are pointing at. You can mark the current face and start a "left cycle" until you return to the marked face. The number of faces you visit is exactly the degree of the vertex, you can easily see this by "flattening" out the part around that vertex, it essentially looks like a flower with either 3, 4 or 5 petals. Now you can easily infer the total number of vertices and then the total number of holes!